

Tap to Ride

Architecture Design Document · Java / Spring Boot Implementation

September 2026



Document Purpose

This document presents the complete system architecture for the Tap to Ride minibus fare collection system. It begins with a high-level overview of the two-process design, then breaks down each internal flow in detail — showing exactly what happens at every step, including the Java classes, database operations, and decision logic involved.

Table of Contents

Section 1. High-Level System Architecture

Section 2. Detailed Flow — Fare Capture (Terminal Side)

Section 3. Detailed Flow — Background Queue & Sync

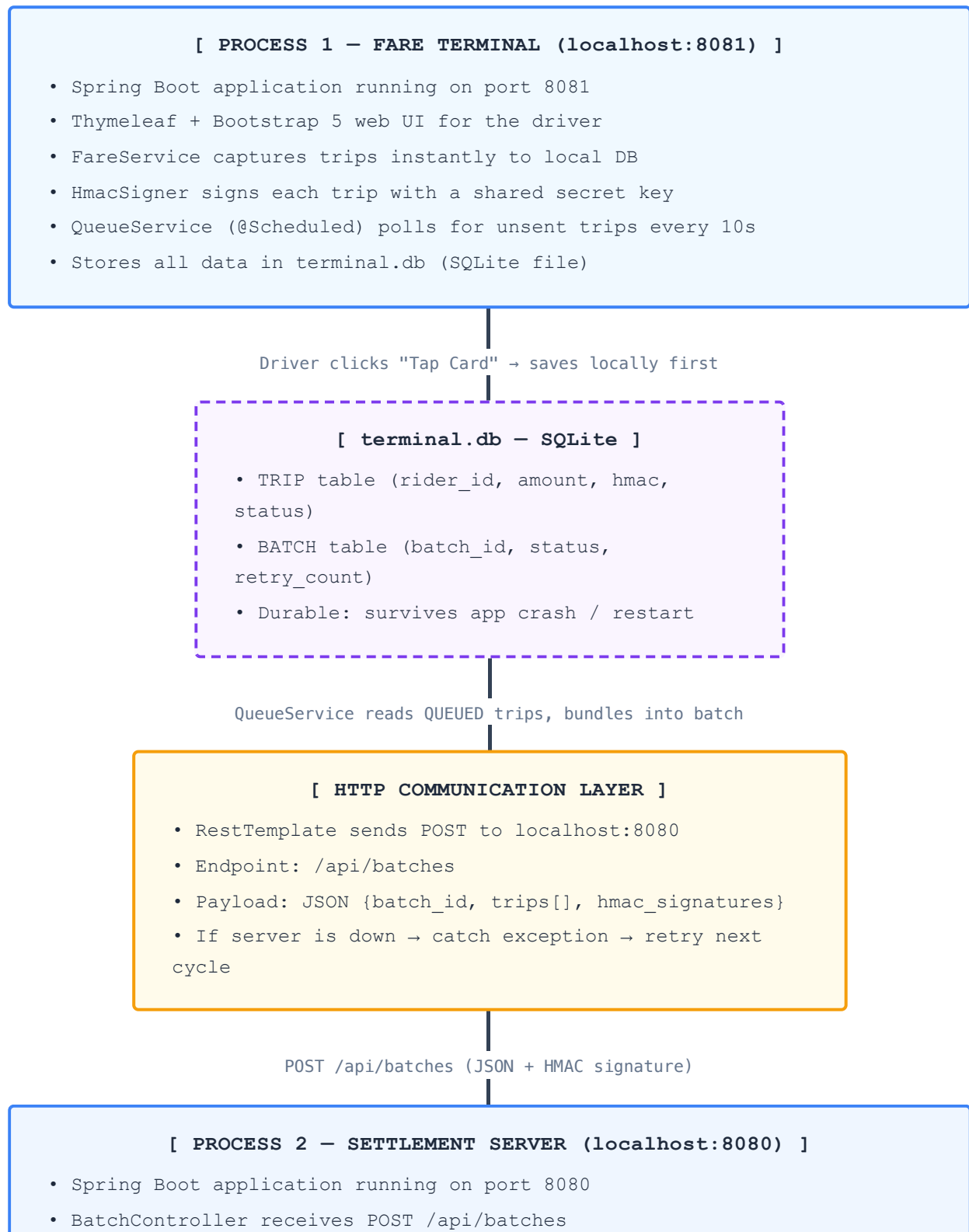
Section 4. Detailed Flow — HMAC Cryptographic Verification (Server Side)

Section 5. Detailed Flow — Idempotency Guard & Settlement Aggregation

Section 6. Database Schemas

Section 1. High-Level System Architecture

The system is split into **two independent Java processes** running on the same machine. They share nothing — each has its own Spring Boot application, its own SQLite database file, and its own port. The Fare Terminal saves trips locally first, then a background thread forwards them to the Settlement Server over HTTP.



- HmacVerifier recalculates signature to detect tampering
- Idempotency Guard rejects duplicate batch_ids
- SettlementService aggregates trips by rider + date
- Operator Console (Thymeleaf) shows reports and totals
- Stores all data in settlement.db (SQLite file)

[settlement.db – SQLite]

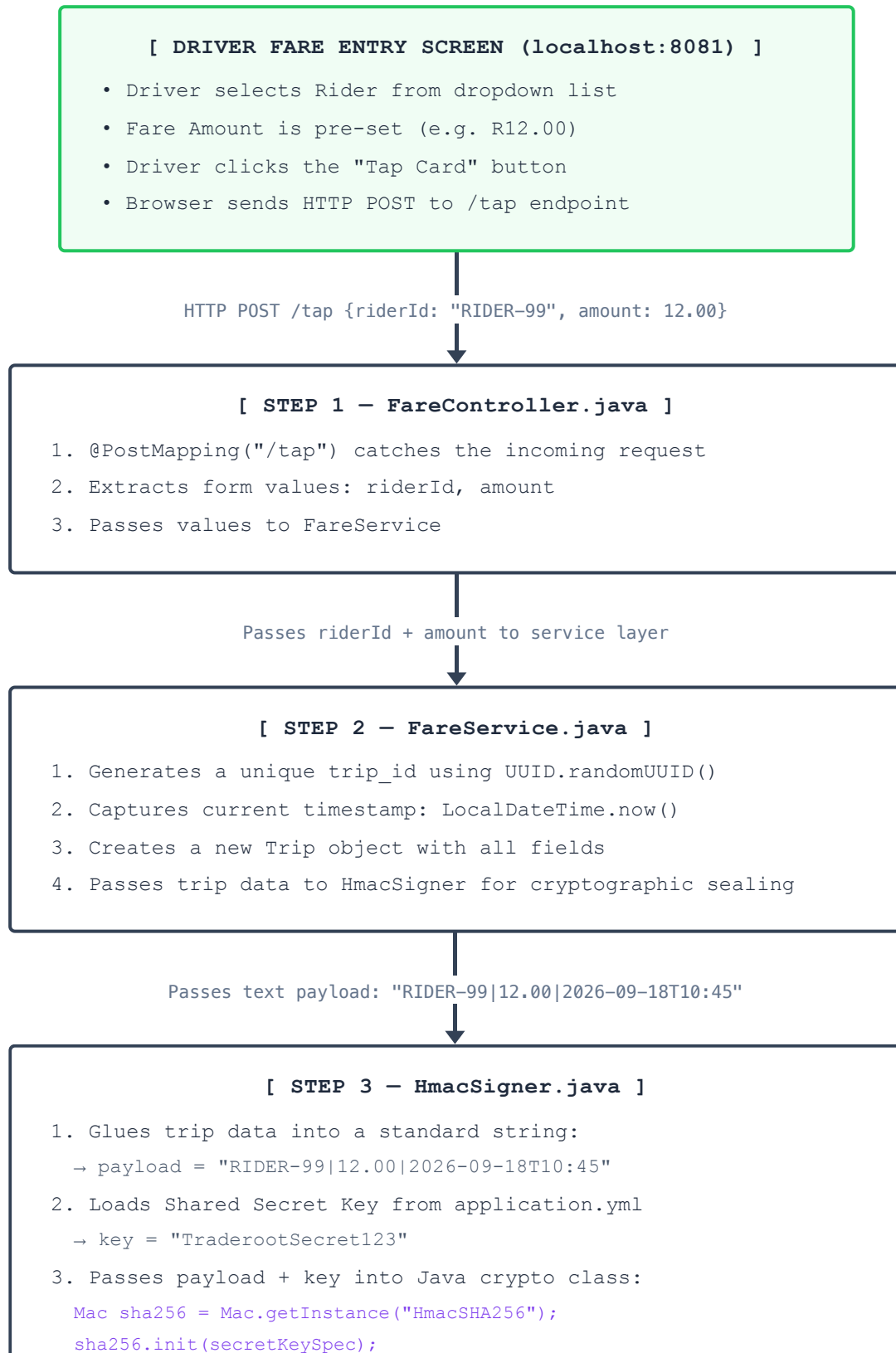
- BATCH table (idempotency key)
- TRIP table (received trips with verification)
- CHARGE table (aggregated daily charges)
- CHARGE_LINE table (trip itemization)

 Core Design Principle

The Fare Terminal NEVER blocks on the Settlement Server. A fare is captured and confirmed in under 1 second regardless of whether the server is online or offline. The two processes are fully decoupled by the local SQLite queue.

Section 2. Detailed Flow — Fare Capture (Terminal Side)

This is the step-by-step breakdown of what happens inside the Fare Terminal when the driver clicks the **"Tap Card"** button. Each numbered step shows the exact Java class and operation involved.



```
byte[] raw = sha256.doFinal(payload.getBytes());
```

4. Converts raw bytes to hex string:
→ Output signature: "xyz789abc..."

Returns signed Trip object (with hmac_signature field set)

[STEP 4 – TripRepository.java (Spring Data JPA)]

1. Spring Data JPA generates SQL command:

```
INSERT INTO trip (trip_id, rider_id, amount, timestamp,  
hmac_signature, status) VALUES (?, ?, ?, ?, ?, 'QUEUED')
```
2. Executes write to local SQLite file
3. Status is set to "QUEUED" (not yet sent to server)

Executes INSERT into local database file

[SQLITE FILE: terminal.db]

- Row written successfully
- Status: "QUEUED"
- Data is durable on disk – survives crash

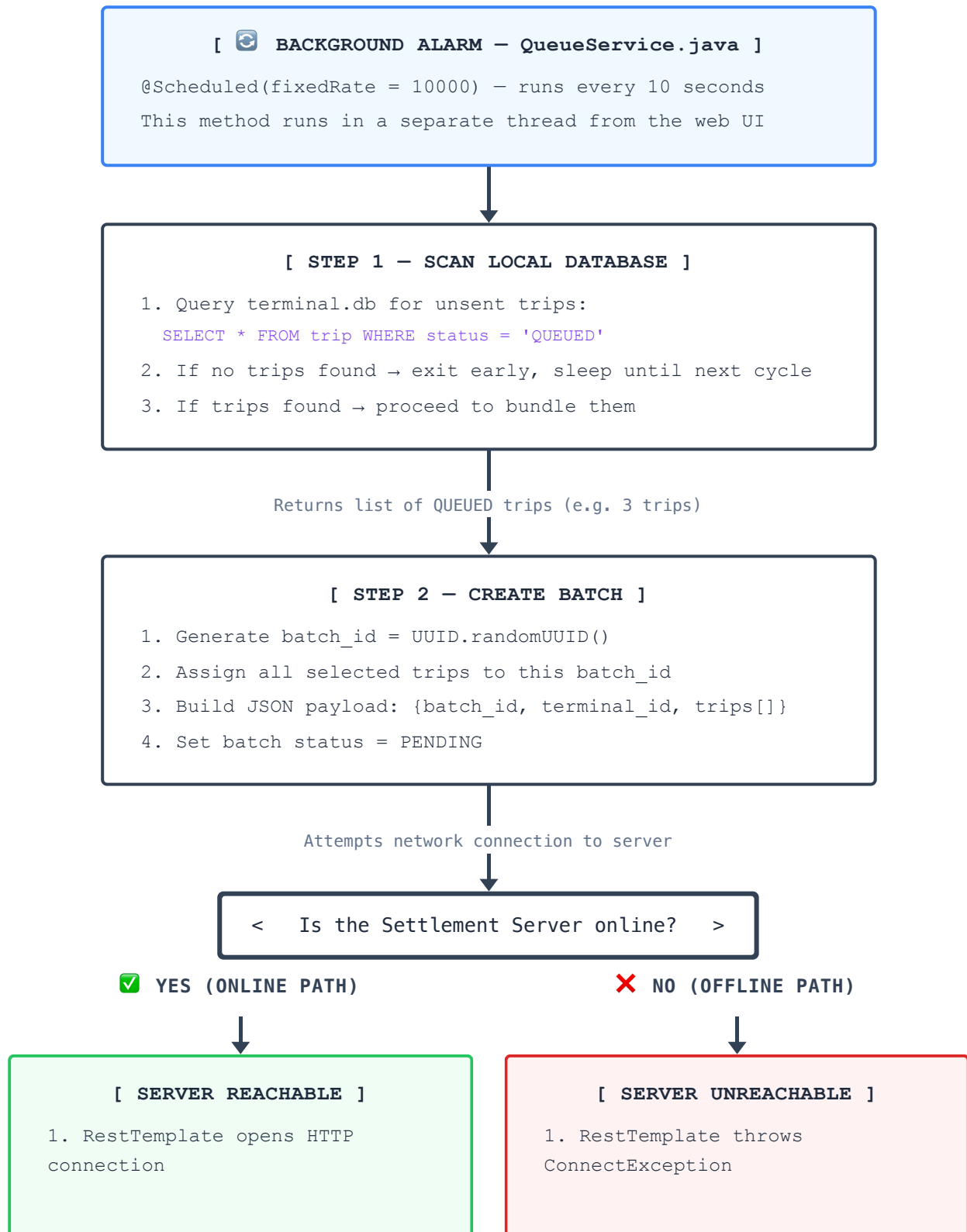
Returns success to controller

[RESPONSE TO DRIVER]

- Green tick confirmation displayed
- Total response time: < 1 second
- Server was never contacted
- Trip sits in local queue, waiting for sync

Section 3. Detailed Flow — Background Queue & Sync

This flow runs **independently** of the driver's actions. A background thread wakes up every 10 seconds, checks for unsent trips, bundles them into a batch, and attempts to send them to the Settlement Server. If the server is offline, the trips remain queued and are retried on the next cycle.



2. Converts batch to JSON using Jackson
3. Sends POST to localhost:8080/api/batches
4. Server processes and returns 200 OK

2. Catch block handles the error gracefully
3. Log warning: "Server offline, will retry"
4. No crash – no error shown to driver

[UPDATE LOCAL STATUS]

- Update trip status → "SENT"
- Update batch status → "SENT"
- Trips cleared from outgoing queue

[QUEUE RETAINED]

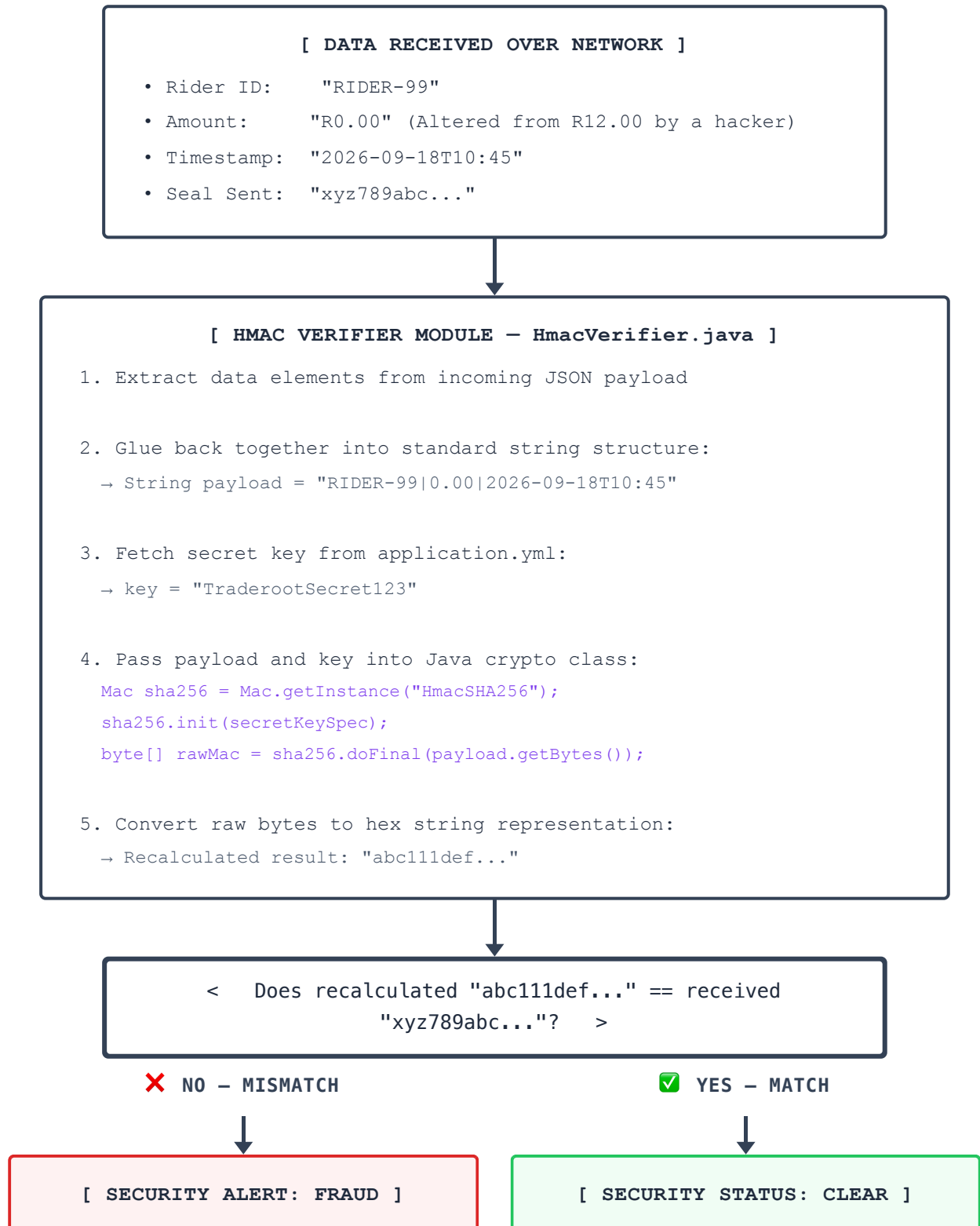
- Trip status stays "QUEUED"
- Batch retry_count incremented
- Data safe inside terminal.db
- Will auto-retry in 10 seconds

⚠ Critical Demo Scenario

During the demo, the evaluator will test what happens when the Settlement Server is shut down mid-shift. The Terminal must continue accepting fares normally. When the server comes back online, all queued trips must automatically sync without any manual action from the driver.

Section 4. Detailed Flow — HMAC Cryptographic Verification

This is the security verification logic that runs inside the Settlement Server when a batch is received. It ensures that trip data has not been altered in transit by recalculating the HMAC signature and comparing it to the one sent by the Terminal.



- Data has been altered in transit
- Trip marked: `signature_valid = false`
- Flagged for investigation
- Trip NOT included in settlement

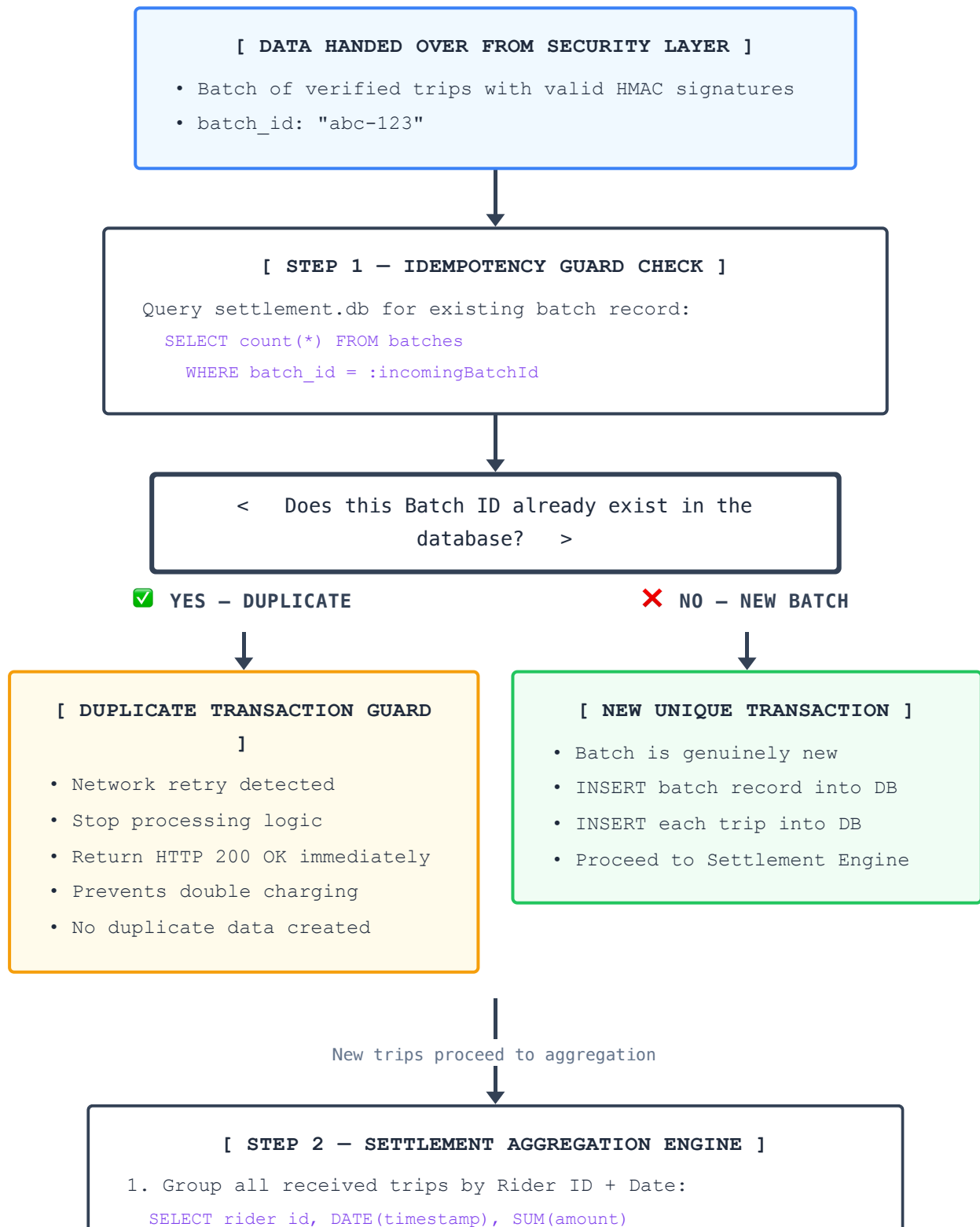
- Signature matches original
- Trip marked: `signature_valid = true`
- Data integrity confirmed
- Forward to Idempotency Guard

Why This Works

The hacker changed the amount from R12.00 to R0.00, but they don't know the shared secret key. When the server recalculates the HMAC using the altered data, it produces a completely different hash — exposing the tampering. Only someone with the secret key can produce a valid signature.

Section 5. Detailed Flow — Idempotency Guard & Settlement Aggregation

This flow runs inside the Settlement Server after a batch passes the HMAC security check. It first checks whether this batch has already been processed (preventing duplicates), then aggregates the trips into daily charges per rider.



```
FROM trips WHERE status = 'RECEIVED'  
GROUP BY rider_id, DATE(timestamp)
```

2. For each rider on that day:
- Calculate total fare amount
 - Create a single CHARGE record
 - Create CHARGE_LINE records linking each trip

[STEP 3 – COMMIT TO STORAGE]

1. Spring Data JPA saves results permanently into settlement.db
2. CHARGE record created with total_amount
3. CHARGE_LINE records itemize each individual trip
4. All trips marked: status = "SETTLED"

[RETURN HTTP 200 OK]

- Response sent back to Terminal
- Terminal marks batch as "SENT"
- Trips cleared from outgoing queue

Settlement Example

Rider	Trip 1	Trip 2	Trip 3	Trip 4	Trip 5	Total Charged
Alice	R12	R12	R12	R12	R12	R60
Bob	R12	R12	—	—	—	R24

Section 6. Database Schemas

6.1 terminal.db (Fare Terminal — Process 1)

The terminal's local database stores trips before they are sent to the server, and tracks batch delivery status.

Column	Type	Description
TRIP Table		
trip_id	TEXT (PK)	UUID — unique identifier for each fare
rider_id	TEXT	e.g. "RIDER_001" — who tapped the card
driver_id	TEXT	e.g. "DRIVER_A" — which driver captured this
amount	DECIMAL	Fare amount, e.g. 12.00
timestamp	DATETIME	When the fare was taken
hmac_signature	TEXT	HMAC-SHA256 hash of trip data
status	TEXT	"QUEUED" (unsent) or "SENT" (delivered to server)
batch_id	TEXT (FK)	NULL until assigned to a batch
BATCH Table		
batch_id	TEXT (PK)	UUID — identifier for the delivery batch
created_at	DATETIME	When the batch was created
status	TEXT	"PENDING", "SENT", or "FAILED"
retry_count	INTEGER	Number of delivery attempts (0, 1, 2...)

6.2 settlement.db (Settlement Server — Process 2)

The server's database stores received trips, tracks batch idempotency, and holds aggregated charges.

Column	Type	Description
BATCH Table		

Column	Type	Description
<code>batch_id</code>	TEXT (PK, UNIQUE)	UUID — idempotency key (prevents duplicates)
<code>terminal_id</code>	TEXT	Which terminal sent this batch
<code>received_at</code>	DATETIME	When the server received it
TRIP Table		
<code>trip_id</code>	TEXT (PK)	UUID — unique trip identifier
<code>batch_id</code>	TEXT (FK)	Links to the source batch
<code>rider_id</code>	TEXT	Who tapped the card
<code>driver_id</code>	TEXT	Which driver captured it
<code>amount</code>	DECIMAL	Fare amount
<code>timestamp</code>	DATETIME	When fare was originally taken
<code>hmac_signature</code>	TEXT	HMAC hash as sent by terminal
<code>signature_valid</code>	BOOLEAN	TRUE if HMAC matches, FALSE if tampered
<code>status</code>	TEXT	"RECEIVED" or "SETTLED"
CHARGE Table		
<code>charge_id</code>	TEXT (PK)	UUID — unique charge identifier
<code>rider_id</code>	TEXT	Rider being charged
<code>charge_date</code>	DATE	The date of travel
<code>total_amount</code>	DECIMAL	Sum of all trips for this rider on this date
<code>created_at</code>	DATETIME	When this charge was generated
CHARGE_LINE Table		
<code>line_id</code>	TEXT (PK)	UUID — unique line item identifier
<code>charge_id</code>	TEXT (FK)	Links to parent CHARGE record
<code>trip_id</code>	TEXT (FK)	Links to the individual TRIP being itemized



Relationship Summary

BATCH → has many TRIPs. CHARGE → has many CHARGE_LINES. Each CHARGE_LINE → references one TRIP. A CHARGE groups all trips for one rider on one day into a single billable record.